Intelligent Robots

LAB6

郝祁 Hao Qi
Email:haoq@sustech.edu.cn

3D Modeling and Simulation

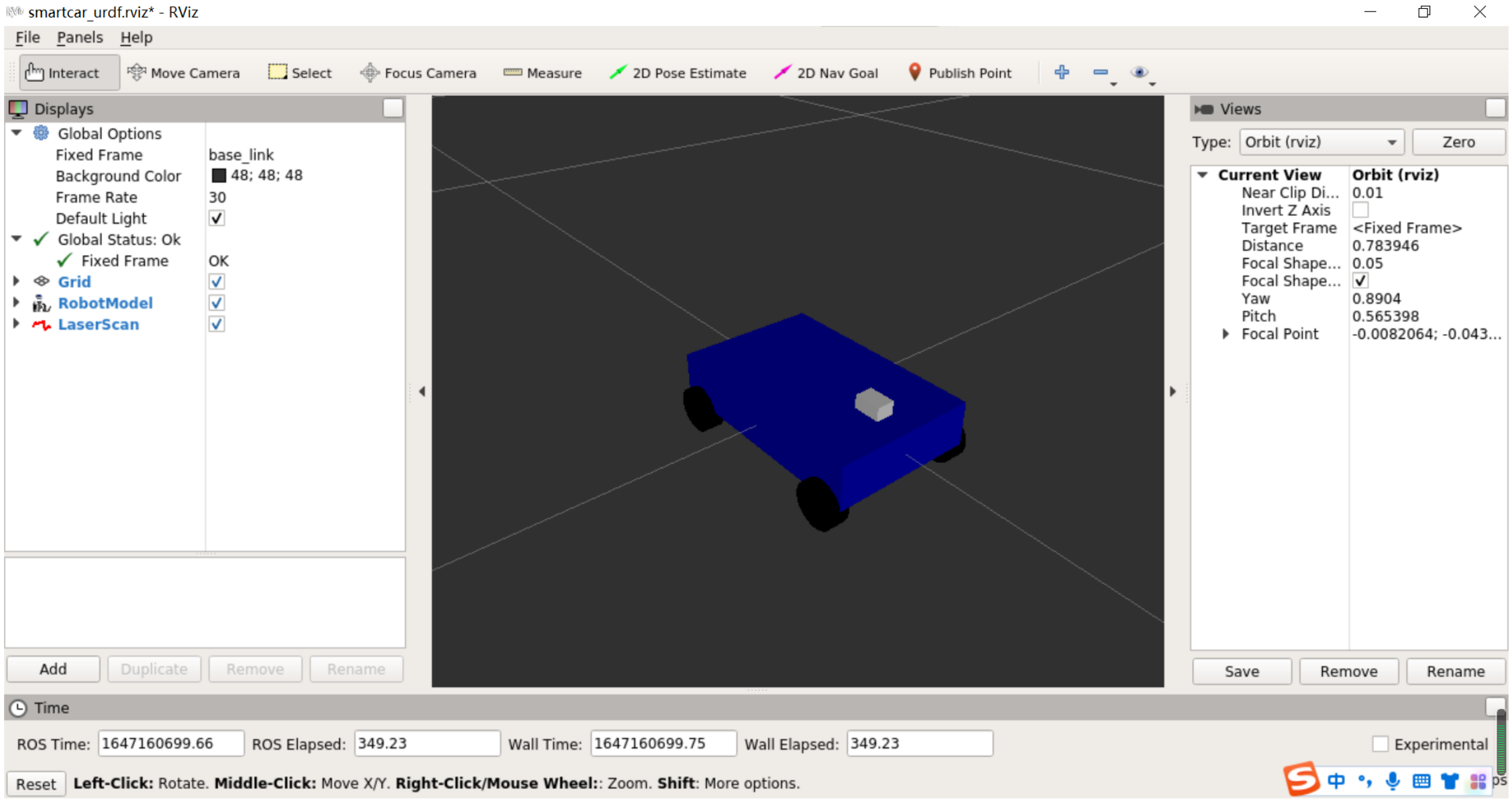
OBJECTIVES

01 Using our 3D model in Gazebo

02 Create a physical simulation environment

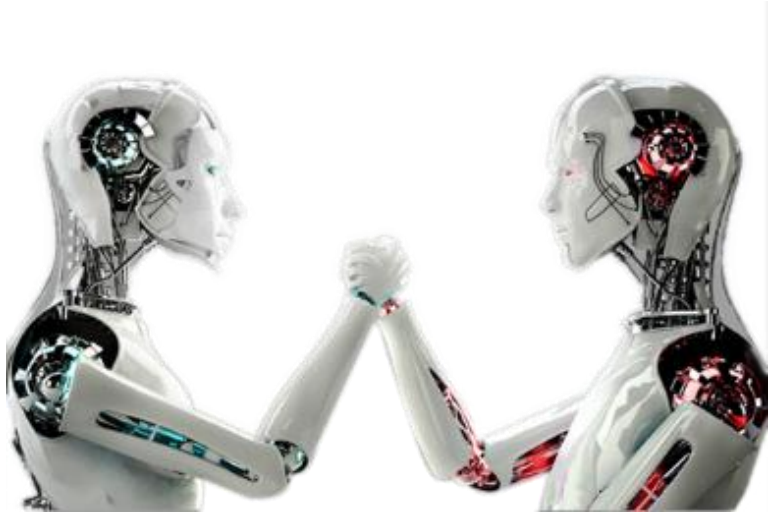
03 Lab Task

Create a smart car based on Gazebo



PART THREE

Using our 3D model in Gazebo



Using our 3D model in Gazebo

1. Add the inertial and collision properties to the link

```
<xacro:macro name="wheel" params="prefix reflectx reflecty reflectz">
    <joint name="${prefix}_wheel_joint" type="continuous">
        <origin xyz="${reflectx*wheel_joint_x} ${reflecty*wheel_joint_y} ${reflectz*wheel_joint_z}" rpy="0 0 0"/>
        <parent link="base_link"/>
        <child link="${prefix}_wheel_link"/>
        <axis xyz="0 1 0"/>
    </joint>

    <link name="${prefix}_wheel_link">
        <visual>
            <origin xyz="0 0 0" rpy="${M_PI/2} 0 0" />
            <geometry>
                <cylinder radius="${wheel_radius}" length = "${wheel_length}"/>
            </geometry>
            <material name="black" />
        </visual>
        <collision>
            <origin xyz="0 0 0" rpy="${M_PI/2} 0 0" />
            <geometry>
                <cylinder radius="${wheel_radius}" length = "${wheel_length}"/>
            </geometry>
        </collision>
        <cylinder_inertial_matrix  m="${wheel_mass}" r="${wheel_radius}" h="${wheel_length}" />
    </link>
```

Using our 3D model in Gazebo

2. Add the gazebo texture to the link

```
<gazebo reference="base_link">
    <material>Gazebo/Blue</material>
</gazebo>
```

```
<gazebo reference="${prefix}_caster_link">
    <material>Gazebo/Yellow</material>
</gazebo>
```

```
<gazebo reference="${prefix}_wheel_link">
    <material>Gazebo/Black</material>
</gazebo>
```

```
<xacro:macro name="mcar_base_gazebo">
    <link name="base_footprint">
        <visual>
            <origin xyz="0 0 0" rpy="0 0 0" />
            <geometry>
                <box size="0.001 0.001 0.001" />
            </geometry>
        </visual>
    </link>
    <gazebo reference="base_footprint">
        <turnGravityOff>false</turnGravityOff>
    </gazebo>

    <joint name="base_footprint_joint" type="fixed">
        <origin xyz="0 0 ${base_height/2 + wheel_radius*2}" rpy="0 0 0" /
        <parent link="base_footprint"/>
        <child link="base_link" />
    </joint>

    <link name="base_link">
        <visual>
```

Using our 3D model in Gazebo

3. Add transmission to the joint

```
<!-- Transmission is important to link the joints and the controller -->
<transmission name="${prefix}_wheel_joint_trans">
    <type>transmission_interface/SimpleTransmission</type>
    <joint name="${prefix}_wheel_joint" >
        <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
    </joint>
    <actuator name="${prefix}_wheel_joint_motor">
        <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
        <mechanicalReduction>1</mechanicalReduction>
    </actuator>
</transmission>
```

Using our 3D model in Gazebo

4. Add the gazebo controller plug-in

```xml
<gazebo>
    <plugin name="differential_drive_controller"
            filename="libgazebo_ros_diff_drive.so">
        <rosDebugLevel>Debug</rosDebugLevel>
        <publishWheelTF>true</publishWheelTF>
        <robotNamespace>/</robotNamespace>
        <publishTf>1</publishTf>
        <publishWheelJointState>true</publishWheelJointState>
        <alwaysOn>true</alwaysOn>
        <updateRate>100.0</updateRate>
        <legacyMode>true</legacyMode>
        <leftJoint>left_wheel_joint</leftJoint>
        <rightJoint>right_wheel_joint</rightJoint>
        <wheelSeparation>${wheel_joint_y*2}</wheelSeparation>
        <wheelDiameter>${2*wheel_radius}</wheelDiameter>
        <broadcastTF>1</broadcastTF>
        <wheelTorque>30</wheelTorque>
        <wheelAcceleration>1.8</wheelAcceleration>
        <commandTopic>cmd_vel</commandTopic>
        <odometryFrame>odom</odometryFrame>
        <odometryTopic>odom</odometryTopic>
        <robotBaseFrame>base_footprint</robotBaseFrame>
    </plugin>
</gazebo>
```

robotNamespace
leftJoint
rightJoint
WheelSeparation
wheelDiameter
commandTopic
odometryFrame

Using our 3D model in Gazebo

5. Load the robot model in gazebo

```
$cd ~/catkin_ws/src
$catkin_create_pkg mcar_gazebo gazebo_plugins gazebo_ros
gazebo_ros_control roscpp rospy
```

- Create the **view_mcar_gazebo_empty_world.launch** file in the **mcar_gazebo/launch** folder, and put the code in it:

Open ▾ view_mcar_gazebo_empty_world.launch
~/[redacted]mcar_gazebo/launch

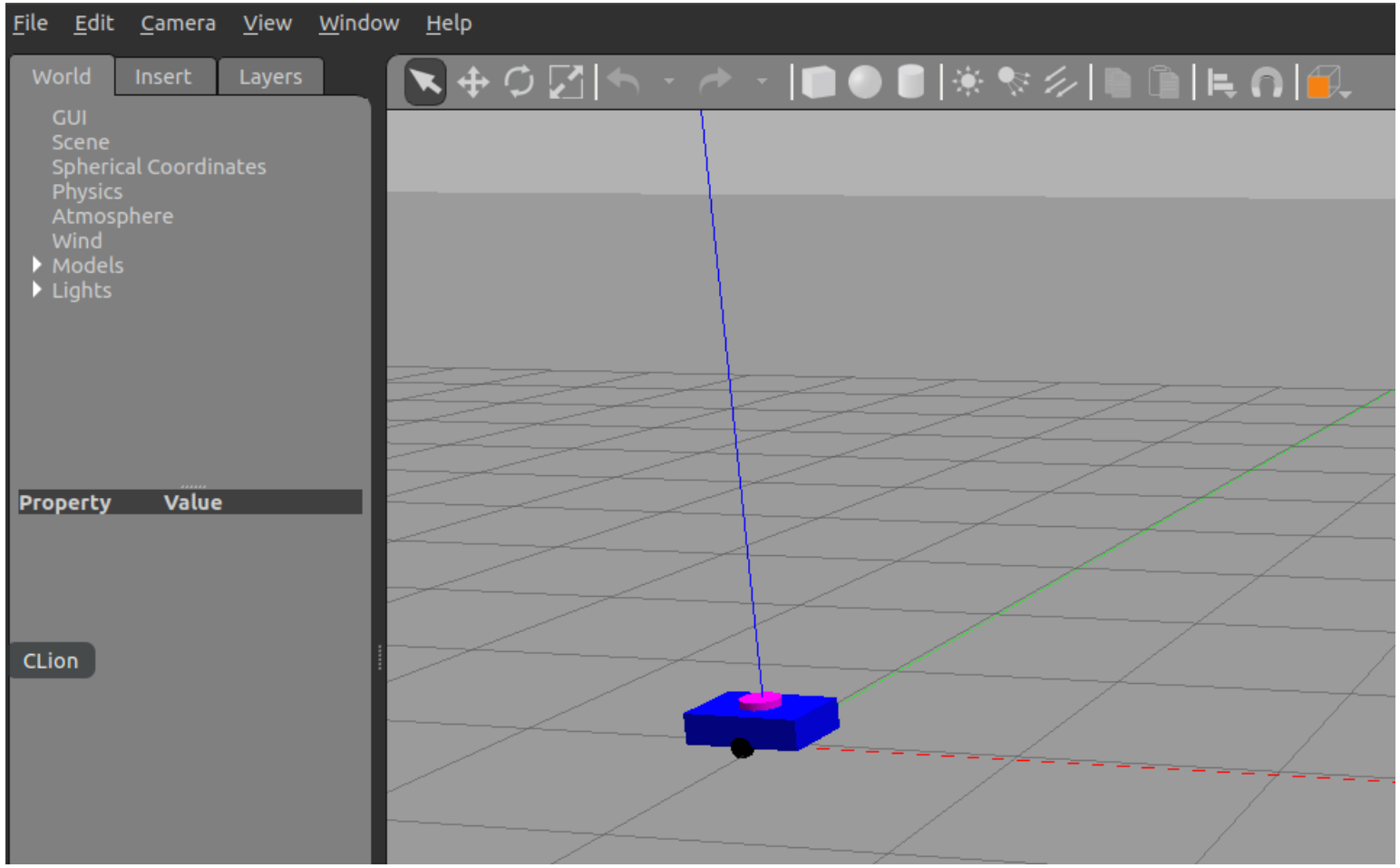
```
<launch>

    <!-- 设置launch文件的参数 -->
    <arg name="paused" default="false"/>
    <arg name="use_sim_time" default="true"/>
    <arg name="gui" default="true"/>
    <arg name="headless" default="false"/>
    <arg name="debug" default="false"/>

    <!-- 运行gazebo仿真环境 -->
    <include file="$(find gazebo_ros)/launch/empty_world.launch">
        <arg name="debug" value="$(arg debug)" />
        <arg name="gui" value="$(arg gui)" />
        <arg name="paused" value="$(arg paused)"/>
        <arg name="use_sim_time" value="$(arg use_sim_time)"/>
        <arg name="headless" value="$(arg headless)"/>
    </include>

    <!-- 加载机器人模型描述参数 -->
    <param name="robot_description" command="$(find xacro)/xacro --inorder '$(find mcar_description)/urdf/mcar_gazebo.xacro'"

    <!-- 运行joint_state_publisher节点，发布机器人的关节状态  -->
    <node name="joint_state_publisher" pkg="joint_state_publisher" type="joint_state_publisher" ></node>

    <!-- 运行robot_state_publisher节点，发布tf  -->
    <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher"  output="screen" >
        <param name="publish_frequency" type="double" value="50.0" />
    </node>

    <!-- 在gazebo中加载机器人模型-->
    <node name="urdf_spawner" pkg="gazebo_ros" type="spawn_model" respawn="false" output="screen"
          args="-urdf -model mrobot -param robot_description"/>

</launch>
```

Using our 3D model in Gazebo

5. Load the robot model in gazebo

- Launch it with the following command:

```
$cd ~/catkin_ws/
$catkin_make
$roslaunch mcar_gazebo view_mcar_gazebo_empty_world.launch
```



Using our 3D model in Gazebo

5. Adding sensors

- kinect simulation
  - <sensor>
  - <camera>
  - <plugin>

```
<link name="${prefix}_frame_optical"/>

<gazebo reference="${prefix}_link">
    <sensor type="depth" name="${prefix}">
        <always_on>true</always_on>
        <update_rate>20.0</update_rate>
        <camera>
            <horizontal_fov>${60.0*M_PI/180.0}</horizontal_fov>
            <image>
                <format>R8G8B8</format>
                <width>640</width>
                <height>480</height>
            </image>
            <clip>
                <near>0.05</near>
                <far>8.0</far>
            </clip>
        </camera>
        <plugin name="kinect_${prefix}_controller" filename="libgazebo_ros_openni_kinect.so">
            <cameraName>${prefix}</cameraName>
            <alwaysOn>true</alwaysOn>
            <updateRate>10</updateRate>
            <imageTopicName>rgb/image_raw</imageTopicName>
            <depthImageTopicName>depth/image_raw</depthImageTopicName>
            <pointCloudTopicName>depth/points</pointCloudTopicName>
            <cameraInfoTopicName>rgb/camera_info</cameraInfoTopicName>
            <depthImageCameraInfoTopicName>depth/camera_info</depthImageCameraInfoTopicName>
            <frameName>${prefix}_frame_optical</frameName>
            <baseline>0.1</baseline>
            <distortion_k1>0.0</distortion_k1>
            <distortion_k2>0.0</distortion_k2>
            <distortion_k3>0.0</distortion_k3>
            <distortion_t1>0.0</distortion_t1>
            <distortion_t2>0.0</distortion_t2>
            <pointCloudCutoff>0.4</pointCloudCutoff>
        </plugin>
    </sensor>
</gazebo>

</xacro:macro>
</robot>
```

Using our 3D model in Gazebo

5. Adding sensors

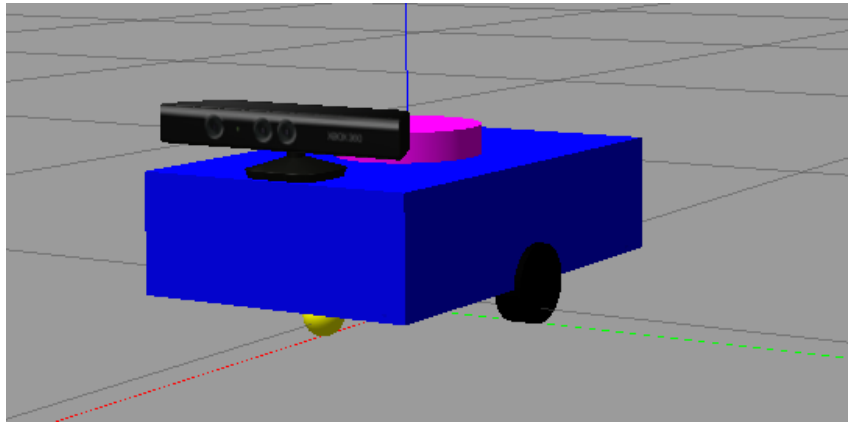
mcar_with_kinect_gazebo.xacro

```xml
<?xml version="1.0"?>
<robot name="arm" xmlns:xacro="http://www.ros.org/wiki/xacro">

    <xacro:include filename="$(find mcar_description)/urdf/mcar_base_gazebo.xacro" />
    <xacro:include filename="$(find mcar_description)/urdf/sensors/kinect_gazebo.xacro" />
    <xacro:property name="kinect_offset_x" value="${base_length/2-wheel_radius}" />
    <xacro:property name="kinect_offset_y" value="0" />
    <xacro:property name="kinect_offset_z" value="0.09" />

    <!-- kinect -->
    <joint name="kinect_joint" type="fixed">
        <origin xyz="${kinect_offset_x} ${kinect_offset_y} ${kinect_offset_z}" rpy="0 0 0" />
        <parent link="base_link"/>
        <child link="kinect_link"/>
    </joint>

    <xacro:kinect_camera prefix="kinect"/>
    <mcar_base_gazebo/>

</robot>
```

Using our 3D model in Gazebo

5. Adding sensors

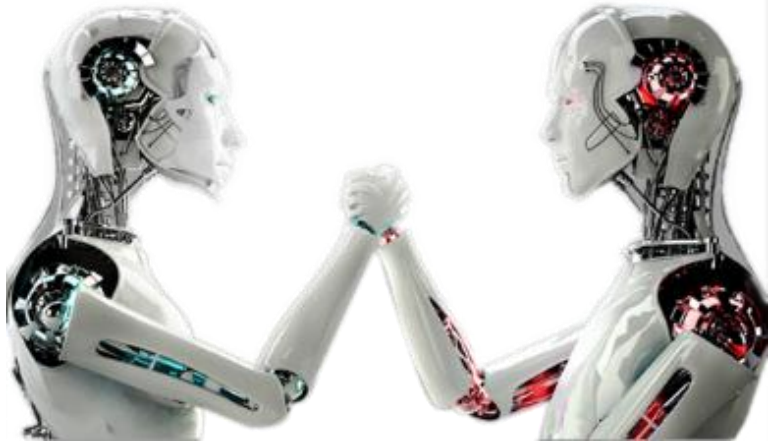
```
$roslaunch mcar_gazebo view_mcar_with_kinect_gazebo.launch
```



```
<launch>

    <!-- 设置launch文件的参数 -->
    <arg name="paused" default="false"/>
    <arg name="use_sim_time" default="true"/>
    <arg name="gui" default="true"/>
    <arg name="headless" default="false"/>
    <arg name="debug" default="false"/>

    <!-- 运行gazebo仿真环境 -->
    <include file="$(find gazebo_ros)/launch/empty_world.launch">
        <arg name="debug" value="$(arg debug)" />
        <arg name="gui" value="$(arg gui)" />
        <arg name="paused" value="$(arg paused)"/>
        <arg name="use_sim_time" value="$(arg use_sim_time)"/>
        <arg name="headless" value="$(arg headless)"/>
    </include>

    <!-- 加载机器人模型描述参数 -->
    <param name="robot_description" command="$(find xacro)/xacro --inorder '$(find mcar_description)/urdf/mcar_with_kinect_gazebo.xacro'" />

    <!-- 运行joint_state_publisher节点，发布机器人的关节状态  -->
    <node name="joint_state_publisher" pkg="joint_state_publisher" type="joint_state_publisher" />

    <!-- 运行robot_state_publisher节点，发布tf  -->
    <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher"  output="screen" >
        <param name="publish_frequency" type="double" value="50.0" />
    </node>

    <!-- 在gazebo中加载机器人模型-->
    <node name="urdf_spawner" pkg="gazebo_ros" type="spawn_model" respawn="false" output="screen"
          args="-urdf -model mrobot -param robot_description"/>
    <!-- 运行depthimage_to_laserscan节点，将点云深度数转换成激光数据 -->
    <node pkg="depthimage_to_laserscan" type="depthimage_to_laserscan" name="depthimage_to_laserscan" output="screen">
        <remap from="image" to="/kinect/depth/image_raw" />
        <remap from="camera_info" to="/kinect/depth/camera_info" />
        <remap from="scan" to="/scan" />
        <param name="output_frame_id" value="/kinect_link" />
    </node>
</launch>
```

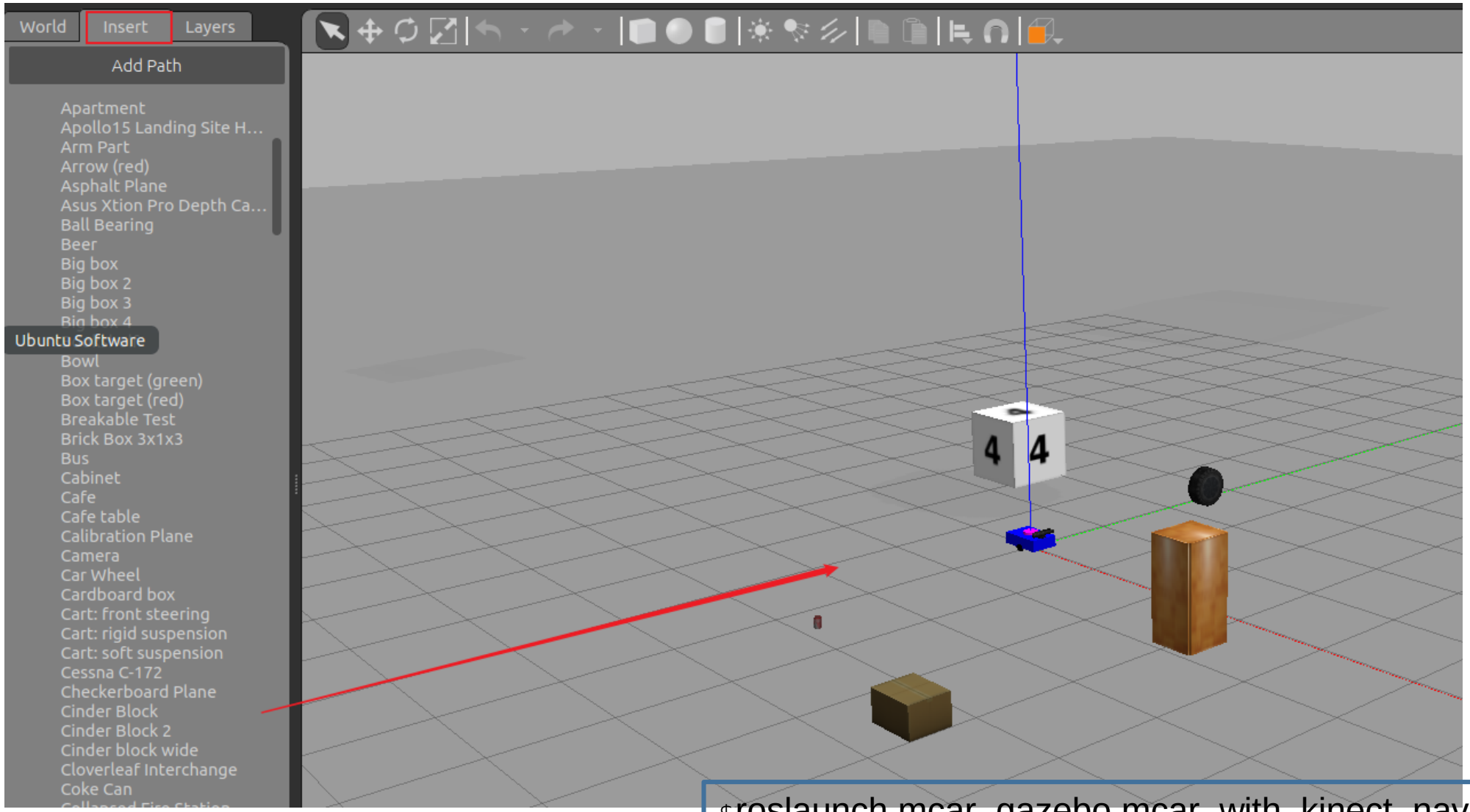
PART FOUR

Create a physical simulation environment



Create a physical simulation environment

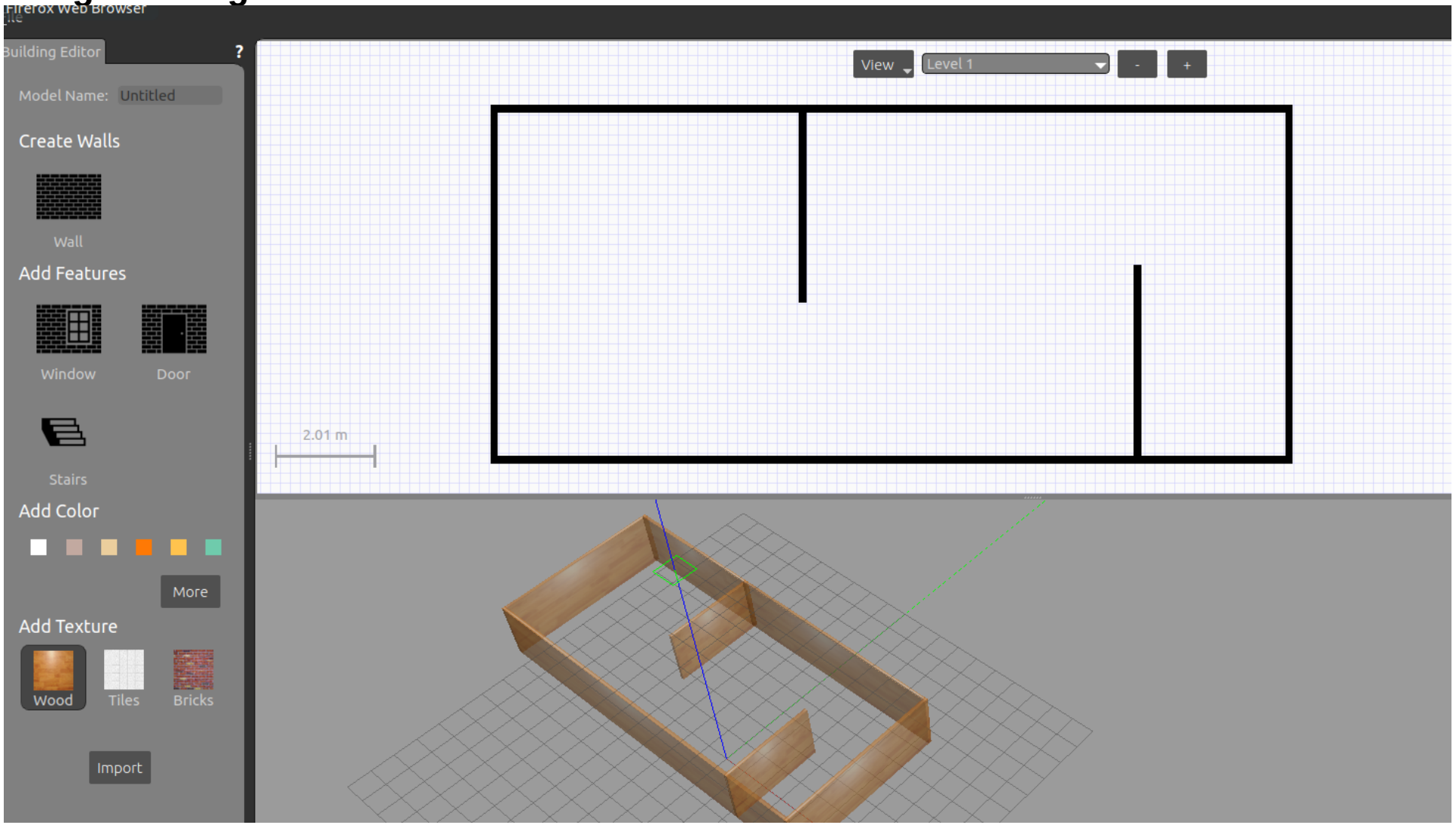
1. Add the environment model using the insert TAB




```
$roslaunch mcar_gazebo mcar_with_kinect_nav_gazebo
```

Create a physical simulation environment

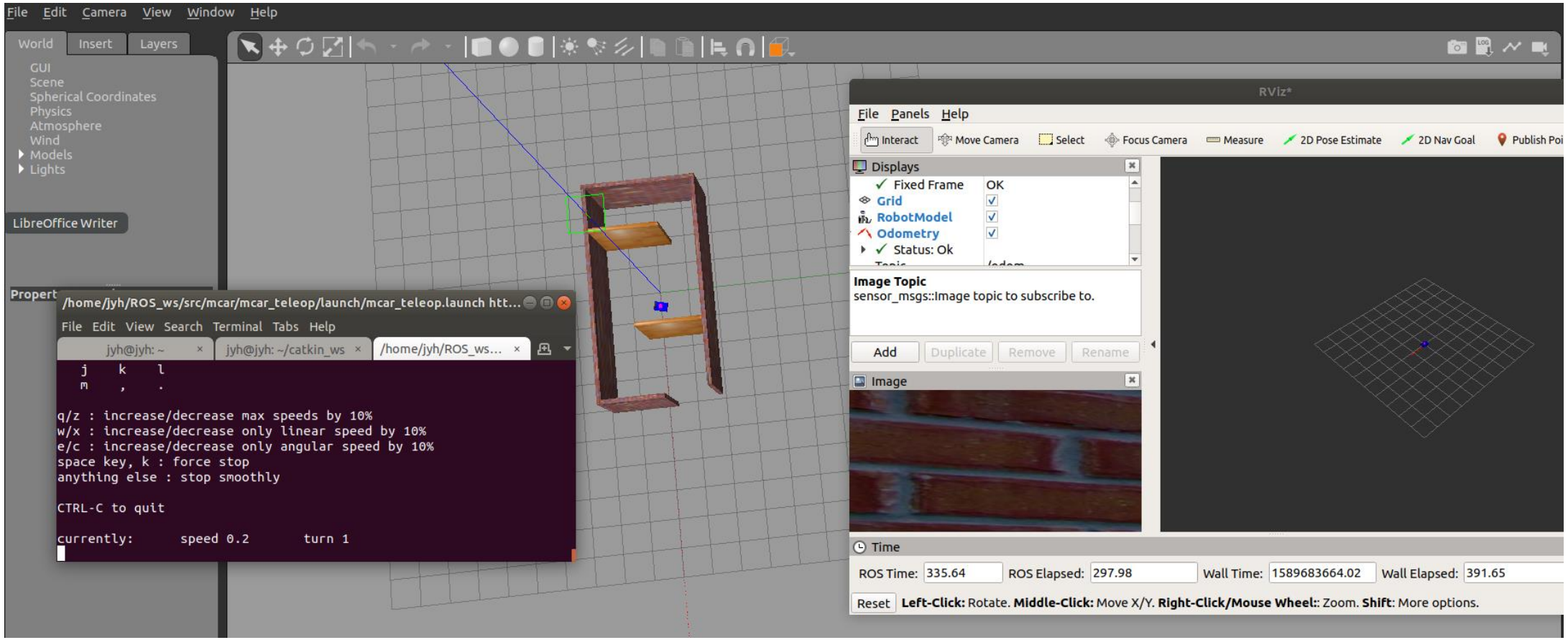
2. using Building Editor



Create a physical simulation environment

3 Launch the keyboard to control the robot movement

```
$roslaunch mcar_telep mbot_teleop.launch
```



Lab Task

- Create a robot model of yourself
- Add a camera and lidar to the robot model , then load the robot model into gazebo, complete the sensor simulation, and display the sensor data in rviz.
- Submit catkin packages and a report to BB,the report contains code, running steps, running results, results analysis, and other contents you want to share!

Lab References

Futher References

- http://wiki.ros.org/urdf
- http://wiki.ros.org/urdf/Tutorials
- http://wiki.ros.org/urdf/XML
- http://wiki.ros.org/xacro
- http://gazebosim.org/tutorials

Thanks